Transactions, MVCC, and Locks

One assignment changes two records.
Two sessions can compete for one row.

Day 1: commit, rollback, and failed changes
Day 2: visible row versions and waiting writers

An assignment has a current state and a history

Record	Before the assignment	Proposed change
Ticket 1004	No assignee, status new	Agent 202 (202), in_progress
Event 5998	No such event	Record who made the assignment

The application needs both changes to persist together. Two successful SQL statements are not automatically one transaction.

Commit and rollback are alternative endings



A later ROLLBACK does not reverse a completed COMMIT.

A complete assignment rehearsal

```
BEGIN;
UPDATE metro_support.tickets
SET assignee_id = 202, status = 'in_progress'
WHERE ticket_id = 1004 AND status = 'new'
RETURNING ticket_id, assignee_id, status;

INSERT INTO metro_support.ticket_events
  (event_id, ticket_id, actor_id, event_type, old_status,
   new_status, note, event_at)
VALUES (5998, 1004, 202, 'status_changed', 'new',
        'in_progress', 'Assigned to Agent 202', now());

ROLLBACK;
```

Start from the supplied fixture. Run the whole block. The lab uses separate practice copies.

A returned row does not establish a commit

Observation	Ticket 1004	Event 5998
Inside the transaction	202, in_progress	Present after the INSERT
After ROLLBACK	NULL, new	Absent
If the same successful pair commits	202, in_progress	Present

```
SELECT ticket_id, assignee_id, status
FROM metro_support.tickets WHERE ticket_id = 1004;
SELECT event_id FROM metro_support.ticket_events WHERE event_id = 5998;
```

A failed statement aborts the open transaction

```
BEGIN;
UPDATE metro_support.tickets
SET priority = 'urgent' WHERE ticket_id = 1004;
INSERT INTO metro_support.ticket_events
SELECT * FROM metro_support.ticket_events WHERE event_id = 5001;
```

The INSERT deliberately duplicates an existing event key.
Expected: unique violation, SQLSTATE 23505.

```
ROLLBACK;
SELECT priority FROM metro_support.tickets WHERE ticket_id = 1004;
```

Run the lower block separately after the error. Expected priority: low.

Zero affected rows is a different result

```
BEGIN;
UPDATE metro_support.tickets SET assignee_id = 202
WHERE ticket_id = 1004 AND status = 'resolved'
RETURNING ticket_id;
ROLLBACK;
```

Ticket 1004 is new. The UPDATE matches nothing, but raises no SQL error.

Application code must inspect the result before writing history or announcing an assignment.

ACID describes different transaction properties

Property	What it means in this case
Atomicity	The assignment and its history commit together.
Consistency	Declared rules and correct application logic preserve valid state.
Isolation	Concurrent reads and writes follow the selected visibility rules.
Durability	A successful commit survives failures covered by the configuration.

Transactions do not retract sent emails. Durable data can still be deleted later, so recovery planning matters.

Lab 1: Keep state and history together

Individual work in transaction_lab, separate from the source tables

Rehearse and commit Agent 201's assignment.
Compare a failed pair with an outdated assignment request.

Read the returned rows before announcing success. Explain why assignment and history must commit together.

Brightspace: week_05_transaction_outcomes.sql, with short SQL comments.

Day 2: One row, competing sessions

A reader can see committed data
while a second writer waits.

The lab compares two ways to end the blocking transaction.

Three connections make the roles explicit

Session A
Changes priority
Leaves work uncommitted

Session B
Changes status
Waits for the same row

Diagnostic connection
Reads activity and blocking relationships

A browser tab is not necessarily a persistent database session. The notebook opens and labels the connections for us.

Notebook fixture: lock_lab.tickets, row 1004 = medium / open.

MVCC gives readers a visible row version

Committed version
1004 / medium / open



An ordinary reader sees
medium / open

A's uncommitted version
1004 / high / open



A sees its own change:
high / open

Multi-version concurrency control preserves versions with visibility rules. Old versions are not a permanent event archive.

Isolation determines what a later read can see

A reads low. B changes the priority to high and commits. A reads again.

A's isolation level	A's second ordinary read	Important boundary
Read Committed	high	Each statement starts a new snapshot.
Repeatable Read	low	Snapshot established by first data statement.
Serializable	low	Conflicting decisions may require a full retry.

The example begins a new transaction for each case. A fresh transaction can see B's committed high value.

A competing update waits for the row

```
BEGIN;
UPDATE lock_lab.tickets
SET priority = 'high'
WHERE ticket_id = 1004;
```

A remains uncommitted

```
BEGIN;
SET LOCAL statement_timeout = '15s';
UPDATE lock_lab.tickets
SET status = 'in_progress'
WHERE ticket_id = 1004;
COMMIT;
```

B waits before COMMIT

The notebook runs this sequence safely in one experiment cell. A timeout in B would not end A's transaction.

PostgreSQL reports both activity and blockers

```
SELECT pid, application_name, state,
       wait_event_type, wait_event,
       pg_blocking_pids(pid) AS blocking_pids,
       clock_timestamp() - xact_start AS transaction_age
FROM pg_stat_activity
WHERE application_name IN
  ('cst4714_session_a', 'cst4714_session_b')
ORDER BY application_name;
```

pid identifies a database process. blocking_pids names the processes preventing that session from continuing.

The notebook narrows this query to this run's two PIDs. Visibility depends on permissions.

An active query can still be waiting

Illustrative PID	Session	State	Wait	Blocking PIDs
7310	A	idle in transaction	ClientRead	{}
7332	B	active	Lock: transactionid	{7310}

B is executing a command, but that command is waiting. A has no current command to run, but its transaction is still open.

Illustrative values matching the supplied trace. Your PIDs and transaction ages will differ.

Both endings release A, but keep different data

A's ending	What B does next	Final priority / status
ROLLBACK	Completes its status update and commits	medium / in_progress
COMMIT	Completes its status update and commits	high / in_progress

```
SELECT ticket_id, priority, status
FROM lock_lab.tickets WHERE ticket_id = 1004;
```

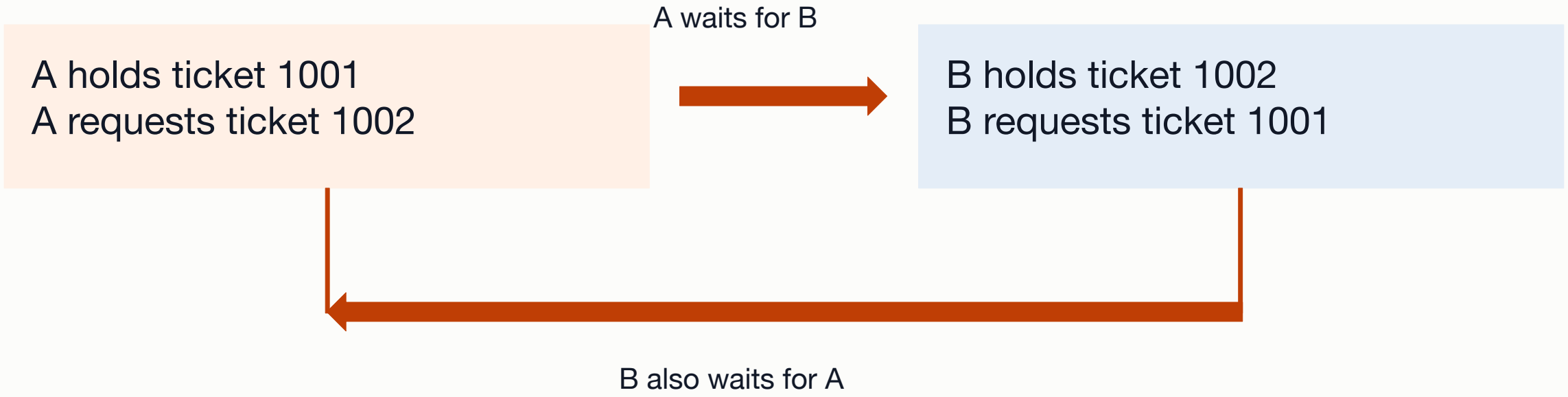
B finishing establishes progress. The row values show which changes survived.

A lock wait and expensive work need different checks

Observation	Useful next check	What it does not establish
Lock wait with a blocker	Owner, transaction state, age, affected operation	That the blocker should be killed
Long execution without this lock wait	Plan, work performed, I/O and other waits	That the query is using CPU efficiently

An index cannot release another session's open transaction. Canceling a query is also different from ending a connection.

A deadlock contains a cycle of waiting



A waits for B, and B waits for A. Neither can finish by waiting alone.

PostgreSQL aborts one transaction. Consistent lock order can prevent this pattern; retries must restart the transaction.

Lab 2 runs in Colab and connects to Supabase

Part	What you use it for
Lab 2	The assignment: compare rollback and commit.
Colab notebook	Runs the supplied Python and SQL cells.
Supabase project	Hosts the PostgreSQL database the cells connect to.

1. Open Lab 2 and click Open in Colab.
2. Choose File > Save a copy in Drive.
3. Keep your existing course project open in Supabase.

Run today's code in Colab. The notebook opens three database connections.

The Session pooler URL supplies the connection

1. In your Supabase project, click Connect.
2. Choose Session pooler and copy the PostgreSQL URI.
3. Replace [YOUR-PASSWORD] with your database password.

Keep the copied host and username. Use the database password.
An API key or Supabase website password will not work here.

```
URL ending before:  /postgres
URL ending after:   /postgres?sslmode=require
```

Password symbols need URL encoding: @ becomes %40 and # becomes %23.
If the URL already has a query, add &sslmode=require instead.

Paste the finished URL only into Colab's hidden connection prompt.

The first run starts with a connection check

```
USE_CLOUD = True
KEEP_A_CHANGE = False  # first run: rollback A
```

Run code cells from the top, one at a time.
In Section 1, paste the URL at the hidden prompt and press Enter.

```
Opened Session A, Session B, and the diagnostic session.
Starting row: (1004, 'medium', 'open')
```

Continue after Section 1 opens the sessions and Section 2 prints the starting row.
A connection error means stop here and check the URL, password, and SSL.

Section 4 runs the experiment. Section 5 shows blockers. Section 6 shows the final row.

Lab 2: Compare two resolutions of the same wait

Individual work in Notebook 02

Run the rollback case and keep its final row.
Predict the commit case, then change the setting and rerun.

In the final Markdown cell: complete the comparison and explain the wait to the developer.

Brightspace: the completed notebook. Run cleanup after each experiment.

A useful incident update explains the data too

The status update waited for A's uncommitted change to the same ticket. The captured result showed B's PID blocked by A's PID. Rolling back A let B finish: priority stayed medium and status became in_progress.

In your comparison, explain how committing A changes that result. State why you could not choose blindly in a real application.

Use your run's observations, or identify the supplied trace and your unexecuted prediction.

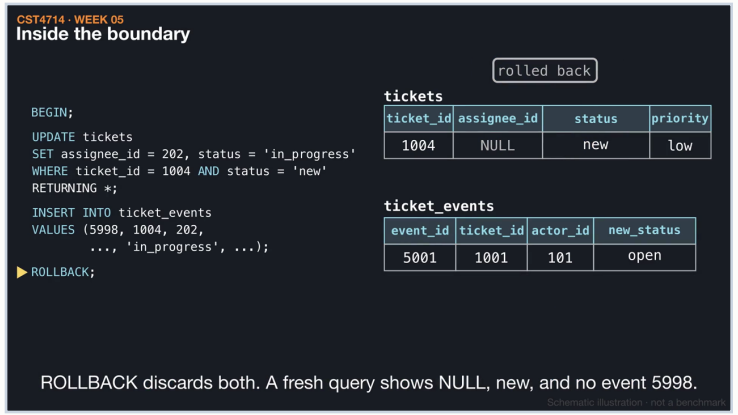
Transaction decisions leave observable results

The database can show us what committed,
what another session could see,
and which transaction prevented progress.

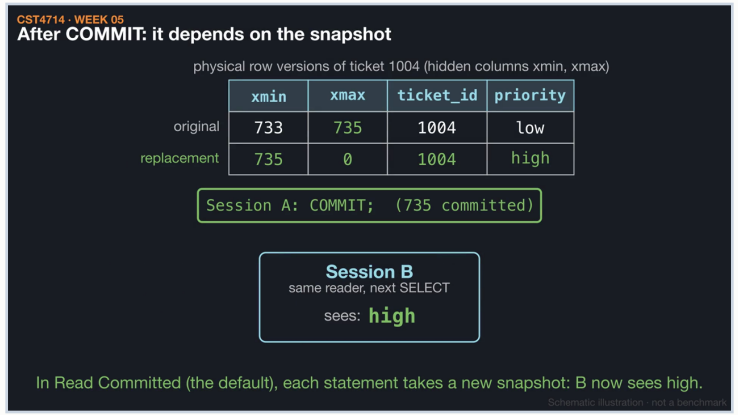
A successful statement, a finished query, and a correct application outcome are different claims.

Next: roles and permissions determine who may perform each operation.

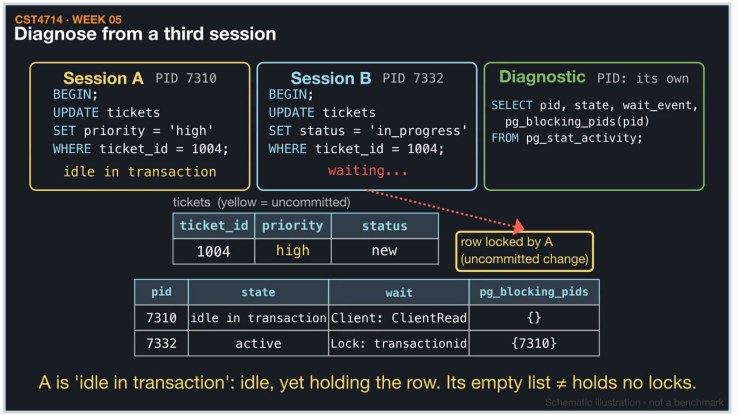
This week's videos



One assignment, one transaction (1:28)



Row versions and snapshots (1:22)



Who is waiting on whom? (1:28)

Click a thumbnail to play on GitHub. Silent, with captions and .srt subtitles; diagrams are schematic.